Homework 2: trust-region methods for real phase retrieval

Group S — Andrés Alarcón Navarro

Question 1 – truncated CG

`tcg(Hv, g, Delta)` solves $\min_{\|v\|\le\Delta} m(v) = \frac{1}{2}\langle v, Hv\rangle + \langle g, v\rangle$ using truncated conjugate gradient method started at $v_0 = 0$, $r_0 = g$, $p_0 = -g$; `Hv` is a *function* $v \mapsto Hv$, so H is never formed. At each iteration, if $\langle p, Hp\rangle \le 0$ or the CG step leaves the ball, we move along p to the boundary, i.e. to the positive root τ of $\|v + \tau p\| = \Delta$ (computed in the cancellation-free form $\tau = -2c/(b + \sqrt{b^2 - 4ac})$ when $b \ge 0$). Otherwise we stop when $\|r\| \le \|g\| \min(\kappa, \|g\|^\theta)$ with $\kappa = 0.1$, $\theta = 1$, or after n iterations. The function also returns Hv (tracked recursively, free of extra products) and $m(0) - m(v)$, which TR-tCG needs. Corner case: for $g = 0$ it returns $v = 0$ (even if H has negative curvature, as for any CG-based solver).

Tests (all in `tests.py`; 19 of 19 pass in the final run; list below, 1-D/2-D/3-D cases have known solutions):

function	what is checked	result
`test_tcg_1d_interior`	tCG 1-D, minimiser strictly inside the ball	pass
`test_tcg_1d_boundary`	tCG 1-D, minimiser outside: stops on the boundary	pass
`test_tcg_1d_negative_curvature`	tCG 1-D, concave model: goes to the correct boundary point	pass
`test_tcg_zero_gradient`	tCG corner case $g = 0$ returns $v = 0$	pass
`test_tcg_zero_hessian`	tCG with $H = 0$: boundary step along $-g$	pass
`test_tcg_tiny_radius`	tCG with tiny Δ: Cauchy-like step $-\Delta g/\|g\|$	pass
`test_tcg_2d_negative_curvature`	tCG 2-D indefinite H, exact solution known	pass
`test_tcg_3d_interior_equals_newton`	tCG 3-D, large ball, tiny tolerance: equals $-H^{-1}g$	pass
`test_tcg_singular_psd_in_range`	tCG 3-D singular PSD H, $g \in \text{range}(H)$	pass
`test_tcg_singular_psd_outside_range`	tCG 2-D singular PSD H, zero curvature along $-g$	pass
`test_tcg_random_properties`	200 random indefinite models: feasibility, Cauchy decrease, tracked Hv	pass
`test_tcg_matrix_free_callable`	tCG only needs a function $v \mapsto Hv$	pass
`test_tr_quadratic`	TR-tCG on a strongly convex quadratic finds $Q^{-1}b$	pass
`test_tr_rosenbrock`	TR-tCG on Rosenbrock reaches $(1, 1)$, f monotone	pass
`test_tr_escapes_saddle`	TR-tCG leaves a saddle point (negative curvature)	pass
`test_tr_radius_rules`	TR-tCG radius updates respect $\bar{\Delta}$ and doubling rule	pass
`test_grad_vs_dense_formula`	gradient equals explicit formula	pass
`test_hessvec_vs_dense_and_symmetry`	Hessian-vector product equals dense Hessian; symmetry	pass
`test_hessvec_vs_finite_difference_of_gradient`	Hessian-vector product vs central differences of ∇f	pass

Question 2 – TR-tCG

`trust_region_tcg(problem, x0, ...)` needs only `cost`, `grad` and `hess_op(x)` (returning $v \mapsto \nabla^2 f(x)[v]$). At x_k it calls `tcg` on $m_k(s) = f(x_k) + \langle \nabla f(x_k), s\rangle + \frac{1}{2}\langle s, \nabla^2 f(x_k)s\rangle$ with radius Δ_k, sets $\rho_k = \frac{f(x_k) - f(x_k + s_k)}{m_k(0) - m_k(s_k)}$ (numerator and denominator are shifted by $10^3 \varepsilon_{\text{mach}} \max(1, |f|)$ against round-off near convergence), accepts the step iff $\rho_k > \rho' = 0.1$, and updates Δ: $\Delta/4$ if $\rho < \frac{1}{4}$; $\min(2\Delta, \bar{\Delta})$ if $\rho > \frac{3}{4}$ and $\|s_k\| = \Delta_k$; unchanged otherwise. Parameters: $\Delta_0 = 1.0$, $\bar{\Delta} = 100$; stop when $\|\nabla f\| \le 1 \times 10^{-8}$. Gradient and Hessian operator are recomputed only after an accepted step. Tests (the four `test_tr_*` rows of the table): a strongly convex quadratic (solution $Q^{-1}b$ in few iterations), Rosenbrock (where $\nabla f, \nabla^2 f$ are known; f decreases monotonically), a saddle that the method must leave via negative curvature, and the radius rules on $f(x) = \|x\|^2$.

Results of the TR-tCG tests. Each test targets one ingredient of the method (all numbers from the last run of `main.py`):

- *Strongly convex quadratic* ($n = 8$, $\Delta_0 = 0.1$). The model is exact, so $\rho \approx 1$ at every step and nothing is rejected (rejections). The first three tCG calls stop on the boundary and Δ doubles ($0.1 \to 0.2 \to 0.4 \to 0.8$); once the ball is large enough, tCG stops on its tolerance and the last call needs $= n$ CG iterations, which is the finite termination of CG. The method returns $Q^{-1}b$ in iterations with error , i.e. it behaves like Newton's method once the radius stops binding.
- *Rosenbrock* (nonconvex, curved valley, $x_0 = (-1.2, 1)$). It converges to $(1, 1)$ in iterations with $\|\nabla f\| =$. steps are rejected ($\rho \le \rho'$), which exercises the shrinking rule, and tCG calls end on negative curvature, so the indefinite Hessian is handled. Since only accepted steps change x, $f(x_k)$ never increases (checked in the test), and Δ never exceeds Δ_0 () because the model is rarely good enough to expand.
- *Saddle* $f = x_1^2 - x_2^2 + \frac{1}{4}x_2^4$, started at $(0.1, 10^{-3})$ next to the saddle at the origin, where plain gradient steps are tiny. TR-tCG uses the negative curvature (call ends on it), leaves the saddle and reaches a minimiser $(0, \pm\sqrt{2})$ within in iterations.

- *Radius rules* on $f = \|x\|^2$ from $x_0 = (10, 0)$ with $\bar{\Delta} = 4$. The model is exact ($\rho = 1$) and the steps hit the boundary, so Δ doubles $1 \to 2 \to 4$ and then stays capped at $\bar{\Delta}$ (maximum radius); the method stops after iterations with $\|\nabla f\| \le 10^{-12}$.

These tests check the *logic* of the method (acceptance, both radius updates, negative curvature, termination) on functions whose derivatives are trivially correct; the correctness of the derivatives of the phase-retrieval objective is verified separately in Question 3, and the behaviour of TR-tCG on that objective is in Question 4.

Question 3 – derivatives of $f(x) = \frac{1}{4n}\|(Ax)^{\odot 2} - y^{\odot 2}\|^2$

Let $z = Ax$ and $r = z^{\odot 2} - y^{\odot 2}$, so $f = \frac{1}{4n}\|r\|^2$. Then $\mathrm{D}z[v] = Av$ and $\mathrm{D}r[v] = 2\,z \odot Av$, hence

$$\mathrm{D}f(x)[v] = \tfrac{1}{2n}\langle r, \mathrm{D}r[v]\rangle = \tfrac{1}{n}\langle r, z \odot Av\rangle = \tfrac{1}{n}\langle r \odot z, Av\rangle = \langle \tfrac{1}{n}A^\top (r \odot z), v\rangle,$$

using $\langle u \odot v, w\rangle = \langle u, w \odot v\rangle$ in the third equality. Therefore

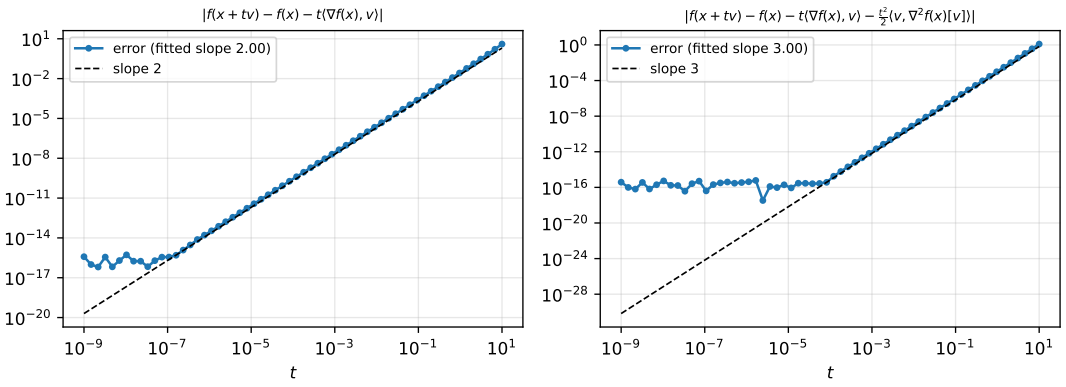
$$\boxed{\nabla f(x) = \tfrac{1}{n}A^\top\big(z^{\odot 3} - y^{\odot 2} \odot z\big)}.$$

Differentiating again, with $\mathrm{D}z[v] = Av$ and $\mathrm{D}r[v] = 2z \odot Av$:

$$\boxed{\nabla^2 f(x)[v] = \tfrac{1}{n}A^\top\big((2z^{\odot 2} + r) \odot Av\big) = \tfrac{1}{n}A^\top\big((3z^{\odot 2} - y^{\odot 2}) \odot Av\big)},$$

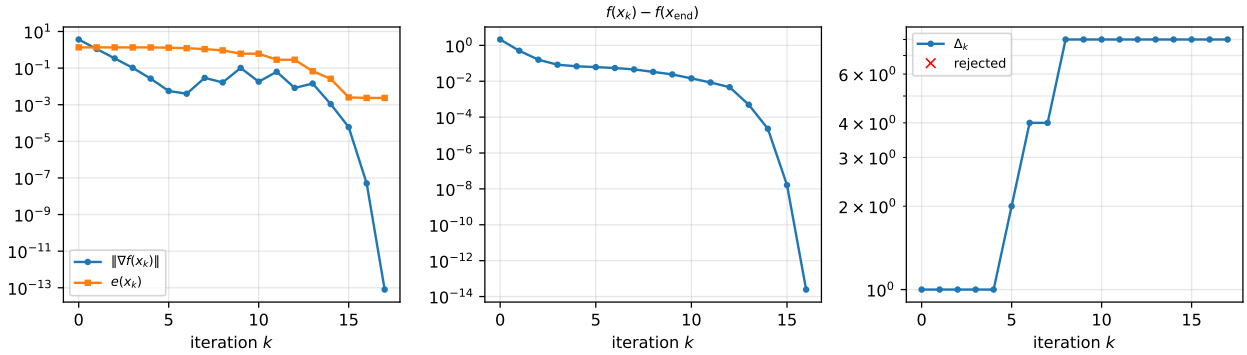
which costs two products with $A/A^\top$ ($O(mn)$) instead of forming the $n \times n$ Hessian $\frac{1}{n}A^\top \mathrm{diag}(3z^{\odot 2} - y^{\odot 2})A$ ($O(mn^2)$). The Hessian-vector product agrees with the dense Hessian to relative error 5.1×10^{-16}; on this machine (one core) one product costs 1.7×10^{-4} s, whereas building the dense Hessian costs 2.8×10^{-3} s ($\approx 16\times$ more at $n = 200$; the gap grows linearly with n).

Numerical checks (`main.py`, figure below): at $x = x_0$ and a random unit v, $|f(x+tv) - f(x) - t\langle \nabla f(x), v\rangle| = O(t^2)$ and $|f(x+tv) - f(x) - t\langle \nabla f(x), v\rangle - \frac{t^2}{2}\langle v, \nabla^2 f(x)[v]\rangle| = O(t^3)$. The fitted log-log slopes are 2.00 and 3.00 (expected 2 and 3), on windows $t \in [10^{-4}, 10^{-2}]$ and $[10^{-3}, 3 \cdot 10^{-2}]$; for smaller t the curves flatten and fluctuate at the round-off level ($\approx \varepsilon_{\mathrm{mach}}|f|$), and for larger t the higher-order terms of the quartic take over. Additional tests: symmetry $\langle u, Hv\rangle = \langle v, Hu\rangle$ and central differences of ∇f.



Question 4 – running TR-tCG

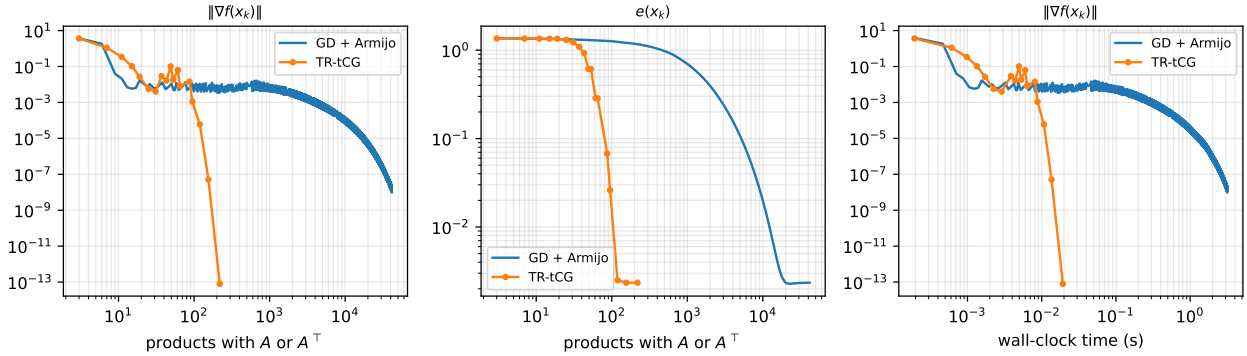
Starting from x_0 (initial error $e(x_0) = 1.36$, $\|\nabla f(x_0)\| = 3.7 \times 10^0$), TR-tCG reaches $\|\nabla f\| \le 1 \times 10^{-8}$ in 17 iterations (218 products with A or $A^\top$, 1.93×10^{-2} s) with $e(x) = 2.35 \times 10^{-3}$; it converges to $-x_{\mathrm{true}}$ (the sign ambiguity), and the final gradient norm is 8.0×10^{-14}. 0 steps were rejected; Δ grew up to 8.0; tCG stopped on tolerance 11 times, on the boundary 5 times, on negative curvature 1 time(s), with at most 31 tCG iterations per call. The gradient norm drops below 10^{-3} at iteration 15 and then collapses (super-linear/quadratic phase). The error plateaus at $\approx 2.35 \times 10^{-3}$: this is the noise floor ($\sigma = 1 \times 10^{-3}$, and the minimiser of f is not x_{true}: $f(x_{\mathrm{true}}) = 8.43 \times 10^{-7} > 5.46 \times 10^{-7} = f(x_{\mathrm{end}})$), not an algorithmic failure.



Comparison with gradient descent (`gradient_descent`: Armijo backtracking, $c_1 = 10^{-4}$, halving, first trial = 2× previous accepted step), same x_0, same stopping rule.

	iterations	products with $A, A^\top$	time (s)	final $e(x)$
TR-tCG	17	218	1.93×10^{-2}	2.35×10^{-3}
GD + Armijo	13839	41517	3.16×10^{0}	2.35×10^{-3}

TR-tCG needs about 190 × fewer products and is about 164 × faster in wall-clock time (814 × fewer iterations, but iterations are not comparable). With the *same budget* of 218 products GD only reaches $\|\nabla f\| = 8.2 \times 10^{-3}$ and $e = 1.2 \times 10^{0}$.



Robustness. Over 10 further random starting points (same A, y; same x_0 for both methods), TR-tCG recovered $\pm x_{\text{true}}$ (error $< 10^{-2}$) in 10/10 runs (14–40 iterations; median 223 products, 1.65×10^{-2} s; worst error 2.35×10^{-3}) and GD in 10/10 runs (median 42460 products, 3.16×10^{0} s).

What should one pay attention to for a fair comparison? (i) Same instance, same x_0, same stopping criterion $\|\nabla f\| \le 1 \times 10^{-8}$, and neither method sees x_{true} (the error is computed afterwards from stored iterates and excluded from the timers). (ii) Count work in a common unit, not iterations: a TR iteration contains several tCG Hessian-vector products (about 91 in total), plus the cost of rejected steps, whereas a GD iteration contains line-search function evaluations; we count *every* product with A or $A^\top$ and also report wall-clock time (both implementations are vectorised, reuse Ax when possible, and use the same counters). (iii) Compare at equal accuracy and in the sign-invariant error $e(x)$, not $\|x - x_{\text{true}}\|$. (iv) One instance and one run is anecdotal and timings are noisy: hence the multi-start medians above.

Question 5 – what did we learn?

1. **The sign ambiguity is real:** TR-tCG converged to $-x_{\text{true}}$, so judging by $\|x - x_{\text{true}}\|$ would have suggested failure; $e(x)$ is the right metric. Incorporating a loss function that accounts for both sign orientations (comparing against $\pm x_{\text{true}}$) proved crucial for accurately evaluating the true performance and convergence of the algorithm.

2. **Derivation nuances & matrix-free mechanics:** Deriving the gradient and the Hessian directly presented its fair share of algebraic friction. Fortunately, the algebraic properties of the Hadamard product (especially shifting element-wise multiplications across the inner product via $\langle u \odot v, w\rangle = \langle u, w \odot v\rangle$) dramatically streamlined these derivative computations. Furthermore, computing the Hessian-vector product $H \cdot x$ directly (rather than ever assembling the full Hessian operator) was an enormous computational save, given the size the Hessian would have. The dense Hessian is about 16 $\times$ more expensive to build at $n = 200$, making an explicit matrix construction unviable. The identity turns the required derivatives into just two matrix-vector products with A and $A^\top$ each. Note that in Taylor validation tests, round-off noise dominates for very small values of t, meaning slopes must be estimated strictly over a suitable window of t.

3. **Conditioning explains the gap with GD:** The entries of A have mean $\frac{1}{2\sqrt{n}} \neq 0$, so $A^\top A$ has one huge eigenvalue (300) and a bulk around 0.47 (condition number 1683). At the computed solution, the Hessian is positive definite with $\lambda_{\min} = 1.42 \times 10^{-4}$, $\lambda_{\max} = 0.4$ (condition number 2930). Because of this severe ill-conditioning, GD restricted to a safe step-size requires thousands of iterations to make progress. In contrast, TR-tCG leverages local curvature information to achieve quadratic-like convergence near the solution.

4. **Details that matter in the implementation:** A robust implementation hinges on three critical edge-cases: employing the cancellation-free formula for the boundary step τ, applying regularization to ρ near convergence to avoid numerical division-by-zero errors, and properly handling the corner case $g = 0$ within tCG. Validating each of these mechanisms individually on low-dimensional tests (1-D/2-D/3-D) with known analytical answers was essential.

5. **Global performance:** All 10 random initializations successfully converged to $\pm x_{\text{true}}$ despite the non-convex landscape. Moreover, tCG required at most 31 iterations per TR step (vastly fewer than the ambient dimension $n = 200$).